

JARVIS

Personal Artificial Executive System — Architecture & Decision Record

Revision: implementation-grounded. Stack and build sequencing folded into the architecture.

Confidence tags [Certain] / [Likely] / [Guessing] appear on engineering claims that carry risk.

What changed in this revision

The original document was architecture without an implementation stack — deliberately, since it scoped stack choices out as build-time decisions. This revision keeps that discipline but adds two things the original left implicit and that you now need because you are building:

1. A Phase 1 stack, scoped to what Phase 1 actually touches — not a forward-looking table that commits to decisions you have not earned the right to make yet.
2. An explicit build-sequencing principle: reliable action execution is proven before the planner is built. This was present in spirit in 'Where This Meets Reality' problem 4; it is now promoted to a build rule.

The language split (security-critical core in a memory-safe systems language; agent layer in Python) is sound and retained. But it is introduced only at the phase where high-consequence capabilities appear, not in Phase 1. In Phase 1 there is no money path and no device path, so the trust boundary guards an empty room and can be enforced in Python at runtime. **[Likely]**

A typed boundary (protobuf/gRPC, or Pydantic in Phase 1) is necessary but not sufficient for security. It validates shape, not provenance flow. The security property — that untrusted-derived requests never reach high-level tools — is runtime logic, enforced by the provenance pre-filter, and is independent of language. **[Likely]**

Implementation Stack

The stack is introduced per phase. A technology appears in this document only at the phase that needs it; earlier rows are deliberately empty. This mirrors the architecture's own rule: capability does not ship ahead of its controls, and tooling is not committed ahead of the need that justifies it.

Stack-selection principles

Enforce the trust boundary at runtime, not by language choice. Memory safety and strong typing reduce one class of bug; they do not enforce that untrusted content cannot reach a high-risk capability. That is the provenance pre-filter's job and it is the same in any language. **[Likely]**

A typed inter-zone request schema is security-critical, but as a carrier of provenance and verifiable fields — not because the serialization format is secure. Its value is that it carries everything verification needs and nothing requiring prose interpretation. **[Likely]**

Defer specialized infrastructure (graph DB, dedicated vector DB, message bus, orchestration) until measured behaviour demands it. A single-user prototype will not produce the contention or scale that justifies them for a long time. **[Likely]**

Iterate the agent layer in Python; harden the executor layer in a systems language only once it holds real capability. Porting a proven Python executor to a typed systems core is mechanical; designing that core before seeing real request traffic produces an underspecified boundary. **[Likely]**

Phase 1 stack (build now)

Concern	Choice	Rationale
Language	Python 3.12+	Fastest iteration; every AI library lands here first
Browser control	Playwright (async)	More reliable than Selenium on modern DOMs
Async runtime	asyncio	Concurrency-ready executor for Phase 2 without a rewrite
Typed requests	Pydantic v2	ActionRequest + provenance enum, runtime-validated
Database	PostgreSQL 16	Battle-tested transactional store for facts/audit
DB access	asyncpg or SQLAlchemy 2 async	Pick one, stay consistent
Vectors	none yet	pgvector added only when retrieval exists
Confirmation	out-of-band minimal	Separate channel from the requesting path
Tests	pytest + pytest-asyncio	Acceptance test per milestone

Async is chosen for Phase 1 despite Phase 1 having no concurrency requirement, so the executor is concurrency-ready for Phase 2. The cost: the reliability harness must manage the event loop correctly, and asyncio plumbing errors can masquerade as browser failures. Mitigation: the harness labels every failure as harness-class or browser-class and treats only browser-class as a reliability data point. [Likely]

Deferred stack (decide later, against a signal)

Technology	Introduced at	Signal that triggers the decision
Systems-language executor core (Rust)	Phase 2–3	First high-consequence capability (money/device) goes live
gRPC + Protobuf inter-zone boundary	Phase 2	Two trust zones exist as separate processes
pgvector	Phase 1–2	Memory retrieval by similarity is actually needed
Dedicated vector DB	much later	pgvector hits real scaling limits
Graph database (Neo4j)	likely never	Relationship traversal becomes the measured bottleneck
Local model routing (vLLM/Ollama)	Phase 2+	Frontier-only calls become the cost bottleneck
Event bus	later	Direct agent-to-agent calls become a debugging problem
Docker Compose → Kubernetes	Phase 3 → much later	Multiple services, then real orchestration need

Picking between vLLM and Ollama, or designing the gRPC schema, before Phase 1's reliability result is in is premature: it answers a question the project has not earned the right to ask. The planner premise is unproven (see Where This Meets Reality, problem 4); committing the expensive parts of the stack ahead of that proof inverts the roadmap's own logic. [Likely]

Build Sequencing Principle — Reliability Before Planner

This is promoted from an observation in the original 'Where This Meets Reality' to a binding build rule, because it determines whether the project is worth continuing.

Every capability that touches the world routes through the Browser and Device agents, and that control is brittle off the shelf today. The first thing built and the first thing tested is therefore reliable browser execution — not the planner, not the agent swarm. [Likely]

The falsification test, stated as the gate:

Can it perform 100 browser tasks in a row at an acceptable failure rate? If no, the rest of the architecture does not matter and the project stops here.

Most Jarvis-style projects fail because the planner is built before reliable action execution exists. Building the expensive, slow-to-iterate security core or the multi-agent planner ahead of a passing reliability test spends the most effort on the least-validated assumption. [Likely]

Phase 1 build order

Each milestone has an acceptance test that must pass before the next begins.

3. Milestone 0 — Browser executor + reliability harness. The gate. Four primitives (navigate/read/fill/click) with bounded retry, plus a harness that runs one task N times and reports failure rate split into browser-class and harness-class errors.
4. Milestone 1 — Typed ActionRequest with provenance label and the escalation/cap rule (untrusted-derived is escalated one level and capped at level 2 automatically).
5. Milestone 2 — Tool router: default-deny allowlist parameterized by provenance. Provenance pre-filters the menu before selection; it is never a post-hoc level bump.
6. Milestone 3 — Memory in Postgres with provenance tags and the Fact Lifecycle expiration field. Stale facts flag, not delete; untrusted writes to durable memory require review.
7. Milestone 4 — Out-of-band confirmation gate for the few level-2 actions: a channel that is not the requesting path, showing DIFF and SOURCE, flagging untrusted-derived triggers red.

Phase 1 explicitly excludes: planner, agent base class / swarm, project graph, world model, trust-zone executor split, gRPC, Rust, graph DB, vector DB, message bus, Kubernetes. When scope is in doubt, the answer is 'later phase — build less.'

Two implementation risks to spike early

Decorrelated verification (Principle 4) is the hardest principle to implement. 'Verify against ground-truth state, not the rationale' is clean on paper, but for a browser action, ground truth means re-reading the page — which is the untrusted surface. Prototype one end-to-end verified action in Phase 1, out of order, to measure what keeping that read out of the kill path actually costs before building on it. [Likely]

Do not build the Agentic Layer's class hierarchy in Phase 1. An 'agent' there is a function that calls a tool. Building multi-agent scaffolding to hold a single browser-calling function repeats, one level down, exactly the premature-structure mistake the planner critique warns against. [Likely]

Architecture (unchanged from the stable design)

The object model below is carried forward unchanged. It reached stability through repeated adversarial review; the revision above adds implementation grounding without altering it. Condensed here for the keep-copy; the principles, trust zoning, project object, authorization framework, and rejection log are the load-bearing content and are preserved in full intent.

Core design principles

- Human remains in control — actions tiered by risk; confirmations resist fatigue (batched, out-of-band, source-revealing).
- Plan before acting — Intent → Plan → Review → Execute → Verify; plan provenance travels to the Review step.
- Continuous world model — user/digital/physical state; every entry carries provenance; untrusted content never writes durable semantic memory without review.
- Verify everything — no agent verifies its own work; verification is decorrelated, checked against ground-truth state, not against the same untrusted input.
- Capability and exposure are separated — the component reading untrusted content is never the component holding high-risk capability. Single most important structural decision.
- Untrusted content is data, never instruction — enforced structurally (separate channels, typed boundaries), not by prompting.
- Default-safe on ambiguity — timeout/ambiguity/conflict/auth-failure resolves to no action.

Trust zoning

Agents split into an untrusted-read zone (Research, inbound Communication, file-read — no direct Level 3+ access) and a privileged-action zone (executors that message, spend, control devices — receive only typed validated requests). The Safety and Auditor agents sit at the boundary and do not ingest raw untrusted content when judging an action derived from it; the verifier checks the typed request against state, not the prose rationale.

Authorization levels

Level	Scope	Confirmation
0	Read-only	None
1	Drafting / recommendations	None
2	Non-destructive actions	Single confirmation
3	Communication / publishing	Strong confirmation
4	Financial / physical actions	Multi-factor, out-of-band
5	Emergency override	Explicit predefined permissions

Provenance gate overrides the table: any action derived from untrusted content is escalated by at least one level, capped at Level 2 automatically, and permission asserted inside content is void. Enforced as a pre-filter on tool selection, not a post-hoc check. Hard ceilings (non-overridable): no transfer above cap without out-of-band MFA; no lock/camera/vehicle

control from untrusted-derived action; no agent writes to vault, audit log, or system config; self-modification routes through this same framework.

Projects — the organizing object

Projects are the persistent object everything hangs off; agents operate on projects, they are not the top-level unit. Each project folds in lifecycle (candidate/active/completed/archived/abandoned), attention (focus/background/dormant), first-class goals, decisions, temporal state (waiting is first-class), reconciliation, and per-project resource budgets enforced by the Runtime Governor. Lifecycle and attention are kept as orthogonal axes. A utility-model scalar score is rejected as false precision; genuine contention is surfaced to the user, not resolved silently in a formula.

Memory and world model

- Tiers: working, episodic, semantic, procedural. Every memory carries provenance (source + trust label + timestamp).
- Fact Lifecycle: semantic facts carry type-keyed expiration; volatile facts flag stale until reconfirmed; stable facts effectively non-expiring. Cheapest high-value control — ship it in Phase 1.
- Credential-bearing data never written to any memory tier; lives only in the vault. Episodic screenshots/logs scrubbed at write time.
- World model graphs (People/Project/Device/Knowledge) carry provenance + confidence. Identity confidence is a People-Graph field, not a layer. User focus mode is declared, not inferred — inferred emotional state is refused as unreliable and a manipulation surface.

Runtime Governor

The dumbest privileged component by design: it counts, pattern-matches, and listens. Three mechanisms — budget meter against rolling ceilings (per-project and global), loop detector over recent action history, progress trigger that terminates when no objective completes in N actions. 'Objective completed' is declared by the goal object via a typed satisfaction event the Governor subscribes to; the Governor never inspects agent output, so it cannot be talked out of killing a runaway.

Security architecture (carried forward)

- Sandboxed execution = the trust boundaries of Principle 5; read-untrusted sandbox cannot call action executors directly.
- Credential vault: injection at the execution boundary; orchestrating model sees placeholders, never raw secrets.
- Audit trail: append-only, externally writable, tamper-evident; not modifiable by any agent.
- Agent-specific controls: instruction/data separation, trust-boundary isolation, tool allowlist + blast-radius ceilings, autonomy budget governors, decorrelated verification, memory-poisoning defense, kill switch (hardware kill for embodied layer), default-safe on failure.

Roadmap (controls ship with capability)

Phase	Capability	Security gate
-------	------------	---------------

1	Conversation, memory, tool calling, browser control	Instruction/data separation, credential non-exposure, tool allowlist, memory provenance, Fact Lifecycle
2	Planning, project mgmt, scheduling, research	Trust-zone isolation, decorrelated auditor, provenance-as-pre-filter ordering, Runtime Governor, Skill Registry
3	Device control, smart home, world model	Level 4 hard ceilings on locks/cameras, provenance gate enforced, autonomy budgets live
4	Vision, gesture, robotics	Hardware kill path, force/rate ceilings, no untrusted-derived actuation
5	Predictive, cross-system orchestration	Full audit tamper-evidence, out-of-band MFA on all Level 4, red-team pass on injection

Where this meets reality (unsolved, by the design's own admission)

8. Reliable browser/computer control — assumed reliability that does not exist off the shelf. This is why Phase 1's gate is the 100x test.
9. Durable memory that does not pollute — provenance/expiration/reconciliation are mitigations, not a proof the line holds over a year.
10. Accurate world models over long horizons — drift compounds; no convincing answer for keeping a personal twin faithful without constant correction.
11. Planning that beats a task queue — the existential one. The multi-agent apparatus is justified only if it materially beats a flat queue with good tools. Unproven; first thing to test.
12. Latency and cost — the machinery that makes the system safe is the same machinery that makes it slow and expensive. Open tradeoff.

The architecture is plausible. Plausible is not proven, and the five problems above are where the proving happens — Phase 1's reliability gate is the first proof attempt.